



CPCS 211: Digital Logic Design

Chapter1 : Introduction

Outline

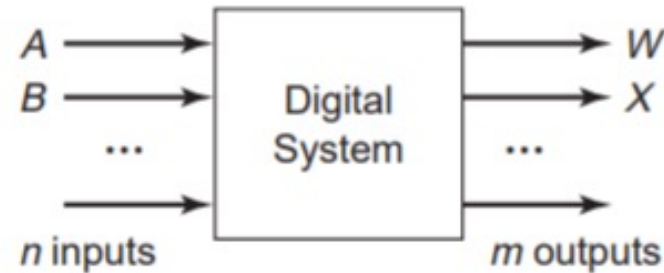
- Logic Design
- Number Systems
- Binary, Octal, Decimal and Hexadecimal Number Systems
- Examples
- Positional Representation
- Number Base Conversion
- Binary Addition, Subtraction and Complements
- Signed and Unsigned Numbers
- Binary Coded Decimal

Digital Systems

- Systems in which all of the signals are represented by discrete values. For example, Computers, Calculators, Digital Counter (Odometer) etc.
- Binary digital systems operate with two values
 - 0 (OFF)
 - 1(ON)

Logic Design

- A digital system may have an arbitrary number of inputs and an arbitrary number of outputs



- **Example:**
A system with three inputs, A , B , and C , and one output, Z , such that $Z = 1$ if and only if (iff) two of the inputs are 1.

Inputs and Outputs

- Inputs and outputs of a digital system represent real quantities
- Sometimes, these are naturally binary, i.e., they take on one of two values
- Other times, they may be multivalued
 - For example, input may be a decimal digit or output might be letter grade for this course
 - Each must be represented by a set of binary digits. This process is referred to as coding the inputs and outputs into binary

Truth Table

- We can describe behavior of a digital system in tabular form
- For the case of the previous example , there are only eight possible input combinations, we can list all of them and what output is for each
- Such a table (referred to as a truth table) is shown

<i>A</i>	<i>B</i>	<i>C</i>	<i>Z</i>
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Number Systems

- A number system is a set of numbers together with one or more operations (e.g. add, subtract)
- Commonly used number systems:
 - Decimal
 - Binary
 - Octal
 - Hexadecimal
- All these number systems are also called weighted number systems

Number systems

- Integers are normally written using a position number system, in which each digit represents the coefficient in a power series as

$$N = a_{n-1}r^{n-1} + a_{n-2}r^{n-2} + \dots + a_2r^2 + a_1r + a_0$$

- where

n is the number of digits

r is the radix or base

a_i are the coefficients

where each is an integer in the range $0 \leq a_i < r$

Example

We can represent 7642 in decimal number systems as

$$(7642)_{10} = 7 \times 10^3 + 6 \times 10^2 + 4 \times 10^1 + 2 \times 10^0$$

Binary Number System (Base-2, Radix-2)

- A different number system; has only two possible values 0 and 1
- Decimal equivalent of a binary number can be found by multiplying coefficients of integral part by increasing powers of 2 (from right to left) and multiplying coefficients of fractional part by decreasing powers of 2 (from left to right) and then adding all these terms as

$$\dots\dots + 2^3 \times a_3 + 2^2 \times a_2 + 2^1 \times a_1 + 2^0 \times a_0 + 2^{-1} \times a_{-1} + 2^{-2} \times a_{-2} + 2^{-3} \times a_{-3} + 2^{-4} \times a_{-4} + \dots$$

where $\dots\dots a_4 a_3 a_2 a_1 a_0 . a_{-1} a_{-2} a_{-3} a_{-4} \dots\dots\dots$ are coefficients of binary number

Powers of 2

- To convert from binary to decimal, just evaluate the power series
- To do that easily, it is useful to know the powers of 2, rather than compute them each time they are needed
- It would save a great deal of time and effort if at least the first 10 powers of 2 were memorized; the first 20 are shown in the Table

n	2^n	n	2^n
1	2	11	2,048
2	4	12	4,096
3	8	13	8,192
4	16	14	16,384
5	32	15	32,768
6	64	16	65,536
7	128	17	131,072
8	256	18	262,144
9	512	19	524,288
10	1,024	20	1,048,576

Binary Integers

We will often be using the first 16 positive binary integers, and sometimes the first 32, as shown in the Table

Decimal	Binary	4-bit	Decimal	Binary
0	0	0000	16	10000
1	1	0001	17	10001
2	10	0010	18	10010
3	11	0011	19	10011
4	100	0100	20	10100
5	101	0101	21	10101
6	110	0110	22	10110
7	111	0111	23	10111
8	1000	1000	24	11000
9	1001	1001	25	11001
10	1010	1010	26	11010
11	1011	1011	27	11011
12	1100	1100	28	11100
13	1101	1101	29	11101
14	1110	1110	30	11110
15	1111	1111	31	11111

Octal Number System (Base-8, Radix-8)

- Has only eight possible values: 0,1,2,3,4,5,6 and 7
- Decimal equivalent of an octal number can be found by multiplying coefficients of integral part by increasing powers of 8 (from right to left) and multiplying coefficients of fractional part by decreasing powers of 8 (from left to right) and then adding all these terms as

$$....+ 8^3 \times a_3 + 8^2 \times a_2 + 8^1 \times a_1 + 8^0 \times a_0 + 8^{-1} \times a_{-1} + 8^{-2} \times a_{-2} + 8^{-3} \times a_{-3} + 8^{-4} \times a_{-4} + ...$$

where $a_4 \ a_3 \ a_2 \ a_1 \ a_0 \ . \ a_{-1} \ a_{-2} \ a_{-3} \ a_{-4}$ are coefficients of octal number

Hexadecimal Number System (Base-16, Radix-16)

- Has sixteen possible values:
0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E and F
- Decimal equivalent of a hexadecimal number can be found by multiplying coefficients of integral part by increasing powers of 16 (from right to left) and multiplying coefficients of fractional part by decreasing powers of 16 (from left to right) and then adding all these terms as

$$\dots + 16^3 \times a_3 + 16^2 \times a_2 + 16^1 \times a_1 + 16^0 \times a_0 + 16^{-1} \times a_{-1} + 16^{-2} \times a_{-2} + 16^{-3} \times a_{-3} + 16^{-4} \times a_{-4} + \dots$$

where a_4 a_3 a_2 a_1 a_0 . a_{-1} a_{-2} a_{-3} a_{-4} are coefficients of hexadecimal number

Numbers in Different Systems

Decimal	Binary	Octal	Hex
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7

Decimal	Binary	Octal	Hex
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Examples

- What is the decimal equivalent of binary numbers $(101111)_2$ and $(101.11)_2$
- What is the decimal equivalent of Octal numbers $(507)_8$
- What is the decimal equivalent of Hexadecimal Number $(10AC.DF)_{16}$

Positional Representation

- The binary number 1101 represents the value

$$V = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

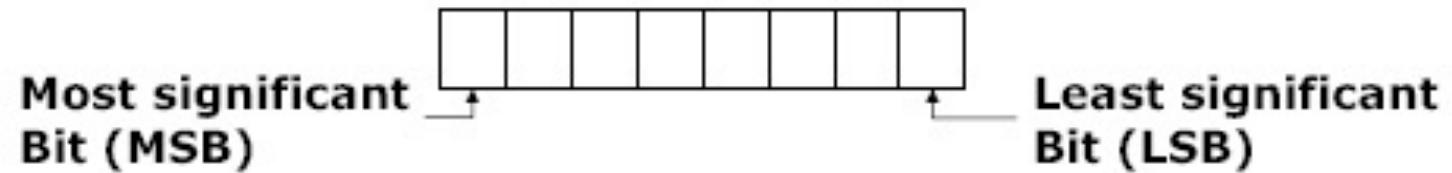
$$V = 8 + 4 + 1 = 13$$

- So

$$(1101)_2 = (13)_{10}$$

- The range of numbers that can be represented by a binary number depends on the number of bits used
- Using *n bits* allows a representation of positive integers in the range 0 to 2^{n-1}

LSB & MSB



Number Base Conversion: *Decimal to Binary*

1. Divide the decimal number by 2 and record the remainder as the right most bit (i.e. least significant bit) of the resulting binary number
2. Divide the quotient obtained in previous step by 2 again and record the remainder as the left bit to the remainder obtained in previous step
3. Repeat the step 2 until the quotient becomes less than the base and record this remainder as the left most bit (i.e. most significant bit) of the resulting binary number

Example

Convert $(857)_{10}$ to binary.

		Reminder
857 ÷	2 = 428	1 LSB
428 ÷	2 = 214	0
214 ÷	2 = 107	0
107 ÷	2 = 53	1
53. ÷	2 = 26	1
26. ÷	2 = 13	0
13. ÷	2 = 6	1
6. ÷	2 = 3	0
3. ÷	2 = 1	1
1. ÷	2 = 0	1 MSB
Result is: $(1101011001)_2$		

Number Base Conversion: *Decimal to Binary*

Use Fraction-Coefficient method

- Fraction is multiplied by the base (i.e. 2) to get an integer and a fraction
 - Record this integer as the first bit to the right of decimal point
- The new fraction is again multiplied by the base to get a new fraction and a new integer
 - Record this new integer as the next bit to the right of previous bit
- This process is continued until the fraction becomes 0 or until the number of digits have sufficient accuracy

Example

Convert $(0.6875)_{10}$ to binary.

Fraction $\times$ Base = Fraction + Coefficient(Integer)

$$0.6875 \times 2 = 0.375 + 1$$

$$0.375 \times 2 = 0.75 + 0$$

$$0.75 \times 2 = 0.5 + 1$$

$$0.5 \times 2 = 0.0 + 1$$

So, $(0.6875)_{10} = (0.1011)_2$

Conversion of Decimal Number to Octal

1. Divide the decimal number by 8 and record the remainder as the right most digit (i.e. least significant digit) of the resulting octal number
2. Divide the quotient obtained in previous step by 8 again and record the remainder as the left digit to the remainder obtained in previous step
3. Repeat the step 2 until the quotient becomes less than the base and record this remainder as the left most digit (i.e. most significant digit) of the resulting octal number

Example

Convert $(624.513)_{10}$ to octal.

We shall first convert 624 to octal as:

		Quotient	Reminder	
624/8	=	78	0	LSD
78/8	=	9	6	
9/8	=	1	1	
1/8	=	0	1	MSD

So, $(624)_{10} = (1160)_8$

Conversion of Decimal Number to Octal

Now, we shall convert the fractional part to octal as:

Fraction $\times$ Base = Fraction + Coefficient(Integer)

$$0.513 \times 8 = 0.104 + 4$$

$$0.104 \times 8 = 0.832 + 0$$

$$0.832 \times 8 = 0.656 + 6$$

$$0.656 \times 8 = 0.248 + 5$$

.....

So, $(0.513)_{10} = (0.4065....)_8$

• Hence $(624.513)_{10} = (1160.4065....)_8$

Conversion of Decimal Number to Hexadecimal

1. Divide the decimal number by 16 and record the remainder as the right most digit (i.e. least significant digit) of the resulting hexadecimal number
2. Divide the quotient obtained in previous step by 16 again and record the remainder as the left digit to the remainder obtained in previous step
3. Repeat the step 2 until the quotient becomes less than the base and record this remainder as the left most digit (i.e. most significant digit) of the resulting hexadecimal number

Example

Convert $(225.225)_{10}$ to hexadecimal.

We shall first convert 225 to hexadecimal as:

	Quotient	Remainder
$225/16$	$= 14$	1 (LSD)
$14/16$	$= 0$	14 i.e. E (MSD)

So, $(225)_{10} = (E1)_{16}$

Conversion of Decimal Number to Hexadecimal

Now, we shall convert the fractional part to hexadecimal as:

Fraction $\times$ Base = Fraction + Coefficient(Integer)

$$0.225 \times 16 = 0.60 + 3$$

$$0.60 \times 16 = 0.60 + 9$$

$$0.60 \times 16 = 0.60 + 9$$

.....

$$\text{So, } (0.225)_{10} = (0.399....)_{16}$$

- Hence $(225.225)_{10} = (E1.399....)_{16}$

Conversion of Binary Numbers to Decimal

- Multiplying every bit of integral part by increasing powers of 2 (RTL)
- Multiplying every bit of fractional part by decreasing powers of 2 (LRT)
- Adding all the results

Example

Convert the binary number $(1011.011)_2$ to decimal.

Solution:

$$\begin{aligned}(1011.011)_2 &= 2^3 \times 1 + 2^2 \times 0 + 2^1 \times 1 + 2^0 \times 1 + 2^{-1} \times 0 + 2^{-2} \times 1 + 2^{-3} \times 1 \\&= 8 \times 1 + 4 \times 0 + 2 \times 1 + 1 \times 1 + 0 + \frac{1}{4} \times 1 + \frac{1}{8} \times 1 \\&= 8 + 0 + 2 + 1 + 0 + 0.25 + 0.125 \\&= 11.375\end{aligned}$$

Conversion of Binary Numbers to Octal

- Make groups of three bits such that the unformed group should be made by writing additional zeros
- For the integral part, make groups starting from RTL
- For fractional part, make groups starting from LTR
- Write the octal equivalent of each group

Example

Convert the binary number $(1111010101111.11111)_2$ to octal.

Solution:

$$(1111010101111.11111)_2 = \textcolor{blue}{00}1\ 111\ 010\ 101\ 111\ .\ 111\ 110$$
$$\qquad\qquad\qquad \textcolor{blue}{1}\quad\textcolor{blue}{7}\quad\textcolor{blue}{2}\quad\textcolor{blue}{5}\quad\textcolor{blue}{7}\quad\textcolor{blue}{7}\quad\textcolor{blue}{6}$$
$$= (17257.76)_8$$

Decimal	Binary	Octal	Hex
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7

Decimal	Binary	Octal	Hex
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Conversion of Binary Numbers to Hexadecimal

- Make groups of four bits such that the unformed group should be made by writing additional zeros
- For the integral part, make groups from right to left (RTL)
- For fractional part, make groups starting from left to right (LTR)
- Write the hexadecimal equivalent of each group

Example

Convert the binary number $(1111010101111.11111)_2$ to hexadecimal.

Solution:

$$(1111010101111.11111)_2 = 0001\ 1110\ 1010\ 1111.1111\ 1000$$

1 E A F F 8

$$= (1EAF.F8)_{16}$$

Decimal	Binary	Octal	Hex
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7

Decimal	Binary	Octal	Hex
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Conversion of Octal Numbers to Binary

Write every octal digit in its binary equivalent by decomposing (breaking) it to a group of three bits

Example

Convert the octal number $(231.4067)_8$ to binary.

Solution:

$$\begin{aligned}(231.4067)_8 &= (010\ 011\ 001.100\ 000\ 110\ 111)_2 \\ &= (010011001.100000110111)_2\end{aligned}$$

Conersion of Octal Numbers to Decimal

- Multiplying every digit of integral part by increasing powers of 8 (RTL)
- Multiplying every digit of fractional part by decreasing powers of 8 (LTR)
- Adding all the results

Example

Convert the octal number $(231.406)_8$ to decimal.

Decimal	Binary	Octal	Hex
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7

Decimal	Binary	Octal	Hex
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Solution:

$$\begin{aligned}(231.406)_8 &= 8^2 \times 2 + 8^1 \times 3 + 8^0 \times 1 + 8^{-1} \times 4 + 8^{-2} \times 0 + 8^{-3} \times 6 \\&= 64 \times 2 + 8 \times 3 + 1 \times 1 + 1/8 \times 4 + 1/64 \times 0 + 512 \times 6 \\&= 128 + 24 + 1 + 0.5 + 0 + 0.0117 = 153.5117\end{aligned}$$

Conversion of Octal to Hexadecimal

1. First convert that octal number to binary by decomposing every octal digit in a group of 3 bits
2. Convert this binary number to hexadecimal number by making groups of four bits

Example

Convert the octal number $(4073.062)_8$ to hexadecimal.

Solution:

We shall first convert octal to binary as

$$(4073.062)_8 = (100\ 000\ 111\ 011\ .\ 000\ 110\ 010)_2$$

Now we shall convert binary to hexadecimal as

$$\begin{aligned}(100000111011.000110010)_2 &= (1000\ 0011\ 1011\ .\ 0001\ 1001\ 0000)_2 \\ &= (83B.19)_{16}\end{aligned}$$

Conversion of Hexadecimal to Binary

Write every hexadecimal digit in its binary equivalent by decomposing (breaking) it to a group of four bits

Example

Convert the hexadecimal number $(A043.DE)_{16}$ to binary.

Solution:

$$(A043.DE)_{16} = (1010\ 0000\ 0100\ 0011.1101\ 1110)_2$$

Conversion of Hexadecimal to Decimal

1. Multiplying every digit of its integral part by the increasing powers of 16 (RTL)
2. Multiplying every digit of fractional part by the decreasing powers of 16 (LTR)
3. Adding all the results

Example

Convert the hexadecimal number $(A043.DE)_{16}$ to decimal.

Solution:

$$\begin{aligned}(A043.DE)_{16} &= 16^3 \times A + 16^2 \times 0 + 16^1 \times 4 + 16^0 \times 3 + 16^{-1} \times D + 16^{-2} \times E \\&= 4096 \times 10 + 0 + 16 \times 4 + 1 \times 3 + 1/16 \times 13 + 1/256 \times 14 \\&= 41027.86\end{aligned}$$

Conversion of Hexadecimal to Octal

1. Convert hexadecimal number to binary by decomposing every hexadecimal digit in a group of 4 bits
2. Convert this binary number to octal number by making groups of 3 bits

Example

Convert the hexadecimal number (A04.DE)₁₆ to octal.

Solution:

We shall first convert hexadecimal to binary as

$$(A04.DE)_{16} = (1010\ 0000\ 0100 . 1101\ 1110)_2$$

A 0 4 D E

$$= (101000000100.11011110)_2$$

Now we shall convert binary to octal as

$$(101000000100.11011110)_2 = (101\ 000\ 000\ 100 . 110\ 111\ 100)_2$$

$$= \quad 5 \quad 0 \quad 0 \quad 4 \quad \quad 6 \quad 7 \quad 4$$

$$= (5004.674)_8$$

Binary Addition

- Common operation required in computers and other digital systems is the addition of two numbers
- To perform binary addition, we shall use the following rules:

$$0+0=0$$

$$0+1=1$$

$$1+0=1$$

$$1 + 1 = 0 \text{ (with 1 carry to the left bit)}$$

Example

Add $(1011)_2$ and $(1001)_2$

Solution:

$$\begin{array}{rcccc} & 1 & & 1 & 1 & \\ & 1 & 0 & 1 & 1 & \text{(Augend)} \\ + & 1 & 0 & 0 & 1 & \text{(Addend)} \\ \hline 1 & 0 & 1 & 0 & 0 & \text{(Sum)} \end{array}$$

Verification

$$\begin{array}{r} 11 \\ + 9 \\ \hline = 20 \end{array}$$

So answer is $(10100)_2$

Overflow

- In a computer with n -bit words, when an arithmetic operation produces a result that is out of range (e.g. addition of n -bit positive integers produces an $(n+1)$ bit result), it is called **overflow**
- When two 4-bit positive integers are added, overflow occurs when the sum is greater than 15 (which is the largest 4-bit integer)
- In the previous example, there was an overflow, because the answer was 20, which is greater than 15

One-bit Adder

- We will denote the carry that is added in as c_{in} and the resulting carry from the addition c_{out}

The addition problem then becomes

$$\begin{array}{r} c_{in} \\ a \\ b \\ \hline c_{out} \ s \end{array}$$

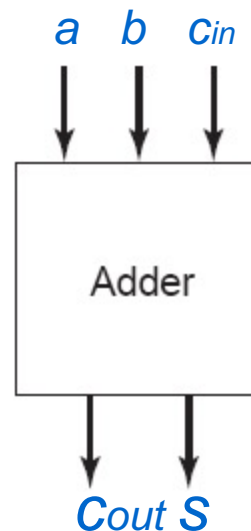


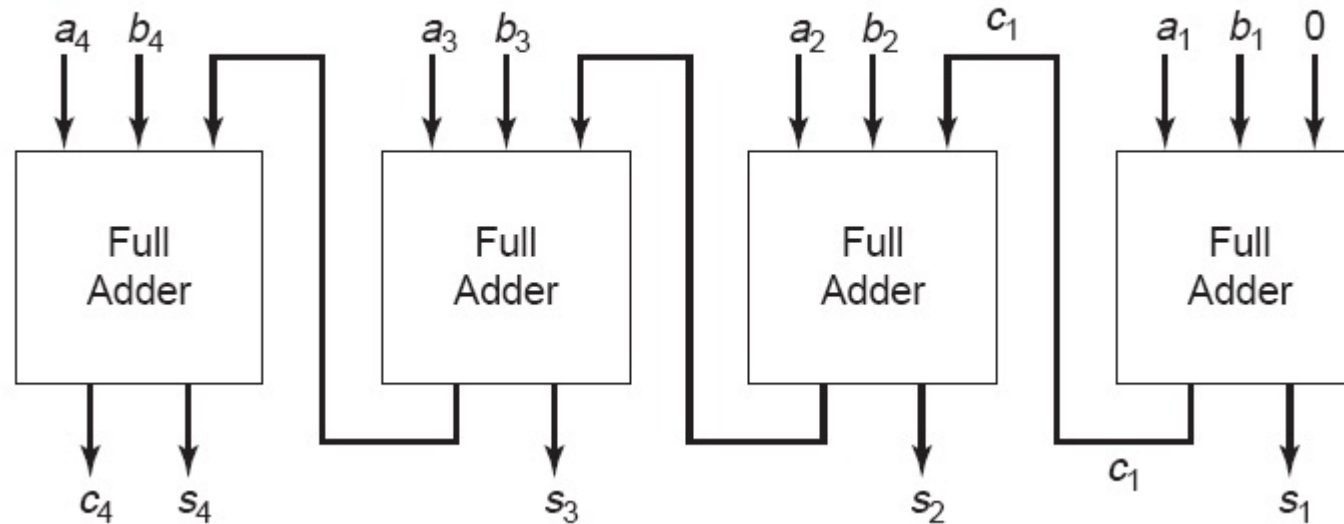
Table 1.5 One-bit adder.

a	b	c_{in}	c_{out}	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

4-bit Adder

In computers, a device that performs the addition of 1-bit is called a **full adder** and is shown as follows:

Figure 1.2 A 4-bit adder.



Binary Subtraction

To perform binary subtraction, we shall use the following rules:

$$0 - 0 = 0$$

$$0 - 1 = 1 \text{ (with borrow from the left bit)}$$

$$1 - 0 = 1$$

$$1 - 1 = 0$$

Example

Subtract $(1011)_2$ from $(11001)_2$

Solution:

					2
	0	0	1	2	
1	1	0	0	1	Minuend
	1	0	1	1	Subtrahend
1	1	1	0	Difference	

Verification

	25
-	11
	14

So answer is $(1110)_2$

Complements

- Complements are used in digital computers for simplifying the subtraction operation and for logical manipulation
- In every base 'r' system, complements are of two types:
 - r's Complement
 - (r-1)'s Complement
- So
 - In decimal number system, we have 10's complement and 9's complement
 - In base-2 number system, we have 2's complement and 1's complement

Complement of Binary Numbers

1's complement of any binary number can be obtained by replacing all 0's by 1's and 1's by 0's in all significant positions

2's complement of any binary number can be obtained by:

- Leaving all least significant 0's unchanged
- Leaving the first non-zero bit unchanged
- Replacing all 0's by 1's and 1's by 0's in all other higher significant positions

Subtracting Using r's Complement

Subtraction of two positive numbers M and N (i.e. $M - N$), both in base 'r', may be obtained as follows

1. Add the minuend 'M' to the r's complement of subtrahend N
2. Inspect the result obtained in step (1) for an end carry
 - If an end carry occurs, discard it
 - If an end carry does not occur, take the r's complement of the result obtained in step (1) and place a negative sign in front of it to get final answer

Important Note: Make the number of digits same in both numbers, M and N by adding additional 0's

Subtraction Using 2's Complement

Using 2's complement, perform $M - N$ with the binary numbers:

$M = (1010100)_2$ and $N = (1000100)_2$

Solution:

$$\begin{aligned} M - N &= M + (\text{2's Complement of } N) \\ &= (1010100)_2 + (\text{2's Complement of } (1000100)_2) \\ &= (1010100)_2 + (0111100)_2 \end{aligned}$$

So, after discarding end carry, final answer is $(0010000)_2$

$$\begin{array}{r} 1010100 \\ + 0111100 \\ \hline 10010000 \end{array}$$

End Carry



Example

Using 2's complement, perform $M - N$ with the binary numbers:

$M = (1000100)_2$ and $N = (1010100)_2$

Solution:

$$\begin{aligned} M - N &= M + (2\text{'s Complement of } N) \\ &= (1000100)_2 + (2\text{'s Complement of } (1010100)_2) \\ &= (1000100)_2 + (0101100)_2 \end{aligned}$$

$$\begin{array}{r} 1\ 0\ 0\ 0\ 1\ 0\ 0 \\ +\ 0\ 1\ 0\ 1\ 1\ 0\ 0 \\ \hline 1\ 1\ 1\ 0\ 0\ 0\ 0 \end{array}$$

Since end carry does not exist, final answer is
 $-(2\text{'s Complement of } (1110000)_2) = -(0010000)_2$

Subtraction Using $(r-1)$'s Complement

Subtraction of two positive numbers M and N (i.e. $M - N$), both in base ' r ', may be obtained as follows:

1. Add the minuend ' M ' to the $(r-1)$'s complement of subtrahend N .
2. Inspect the result obtained in step (1) for an end carry.
 - If an end carry occurs, add it to the LSB of the result obtained.
 - If an end carry does not occur, take the $(r-1)$'s complement of the result obtained in step (1) and place a negative sign in front of it to get final answer.

Important Note: Make the number of digits same in both numbers, M and N by adding additional 0's

Subtraction Using 1's Complement

Using 1's complement, perform $M - N$ with the binary numbers:

$M = (1010100)_2$ and $N = (1000100)_2$

Solution:

$M - N = M + (1's \text{ Complement of } N)$

$$= (1010100)_2 + (1's \text{ Complement of } (1000100)_2)$$

$$= (1010100)_2 + (0111011)_2$$

So, final answer is $(0010000)_2$

$$\begin{array}{r} 1010100 \\ + 0111011 \\ \hline 1110111 \\ + 1 \\ \hline 0010000 \end{array}$$

End Carry

Example

Using 1's complement, perform $M - N$ with the binary numbers:

$M = (1000100)_2$ and $N = (1010100)_2$

Solution:

$$\begin{aligned} M - N &= M + (2\text{'s Complement of } N) \\ &= (1000100)_2 + (1\text{'s Complement of } (1010100)_2) \\ &= (1000100)_2 + (0101011)_2 \end{aligned}$$

$$\begin{array}{r} 1\ 0\ 0\ 0\ 1\ 0\ 0 \\ +\ 0\ 1\ 0\ 1\ 0\ 1\ 1 \\ \hline 1\ 1\ 0\ 1\ 1\ 1\ 1 \end{array}$$

Since end carry does not exist, final answer is
- (1's Complement of $(1101111)_2$) = - $(0010000)_2$

Comparision Between 2's and 1's Complement

- 1'sC easily obtained, only change all 1's into 0's and all 0's into 1's.
- During subtraction, 2'sC is advantageous, only one arithmetic addition operation is required whereas in 1'sC need to perform two arithmetic additions, when an end carry exists.
- 1'sC has disadvantage that it has to represent 0 in two ways: one with all 0's and other with all 1's. (For example, consider $(1100)_2 - (1100)_2$)
- Hence 2'sC has advantage over 1'sC because it represents 0 in only one way, but 2'sC can be obtained by finding 1'sC of the number and then adding 1 in the result.

Signed Numbers

- Up to this point, we have only considered positive integers, *unsigned numbers*
- Computers must deal with *signed numbers*, that is, both +ve and -ve numbers
- Could be incorporated using first bit of a number as a *sign indicator* (normally 0 for *positive* and 1 for *negative*) and the remaining bits for the *magnitude*
- In a 4-bit system, we would represent: +5 $\rightarrow$ 0101, -5 $\rightarrow$ 1101, -3 $\rightarrow$ 1011
- With 3 bits for magnitude, the range of numbers available would be from -7
- to +7. (i.e. 1111 for -7 and 0111 for +7)
- Major problem with *signed-magnitude* is complexity of arithmetic

Signed Numbers

Consider the following addition problems:

$\begin{array}{r} +5 \\ +3 \\ \hline +8 \end{array}$	$\begin{array}{r} -5 \\ -3 \\ \hline -8 \end{array}$	$\begin{array}{r} +5 \\ -3 \\ \hline +2 \end{array}$	$\begin{array}{r} -5 \\ +3 \\ \hline -2 \end{array}$	$\begin{array}{r} -3 \\ +5 \\ \hline +2 \end{array}$	$\begin{array}{r} +3 \\ -5 \\ \hline -2 \end{array}$
--	--	--	--	--	--

- In the first two, signs of the two operands are the same, just add magnitudes and retain sign; computation is $5 + 3$.
- In each of the other examples, must subtract smaller from larger, then attach the sign of the larger magnitude; computation is $5 - 3$.

Signed Numbers

- Signed binary numbers are nearly always stored in 2'sC format
 - The left most bit is still the sign bit (0 for positive and 1 for negative).
- Positive numbers (and zero) are just stored in normal binary
- The largest number that can be stored is $2^{n-1} - 1$
 - *(e.g. in 4-bit system, largest number that can be stored is 7 or +7 because it can be written as 0111). Thus, in a 4-bit system, 5 would be stored as 0101.*

Signed Numbers

- The negative number, $-a$, is stored as the binary equivalent of $2^n - a$ in an n -bit system
- Thus, for example, -3 is stored (in 4-bit system) as the binary for $2^4 - 3 = 16 - 3 = 13$, that is, 1101 (because 2'sC of 3 i.e. 011 is 101, omitting the left most signed bit).
- The smallest negative number that can be stored is $-2^n - 1$ (i.e. -8 in a 4-bit system)

Signed Numbers

An easier way to find the **storage format** for negative numbers in 2'sC is the following three-step process:

1. Find the binary equivalent of the magnitude only
2. Complement each bit (that is, change 0's to 1's and 1's to 0's)
3. Add 1

Example

To find the storage format of -5, proceed as

- Binary equivalent of 5 is 0101 (in 4-bit system)
- Complementing each bit, we get 1010
- 3. Adding 1, get 1011 (for -5)
 - As 2'sC of 101 is 011 and left most bit i.e. 1 is for negative sign

In other words, in order to represent the storage format of negative number, just take 2'sC of the binary equivalent of the magnitude and place 1 at the left most position.

Another Example

To find the storage format of -1, we proceed as

1. Binary equivalent of 1 is 0001 (in 4-bit system)
2. Complementing each bit, we get 1110
3. Adding 1, get 1111 (for -1)

As 2'sC of 001 is 111 and left most bit i.e. 1 is for negative sign

Another Example

To find the storage format of -0, we proceed as

1. Binary equivalent of 0 is 0000 (in 4-bit system)
2. Complementing each bit, we get 1111
3. Adding 1, get 0000 (for -0, ignoring end carry)

Signed and Unsigned 4-bit Numbers

Following Table lists the meaning of all 4-bit numbers both as positive (unsigned) numbers and as two's complement signed numbers

Table 1.6 Signed and unsigned 4-bit numbers.

Binary	Positive	Signed (two's complement)
0000	0	0
0001	1	+ 1
0010	2	+ 2
0011	3	+ 3
0100	4	+ 4
0101	5	+ 5
0110	6	+ 6
0111	7	+ 7
1000	8	- 8
1001	9	- 7
1010	10	- 6
1011	11	- 5
1100	12	- 4
1101	13	- 3
1110	14	- 2
1111	15	- 1

2's Complement of 010 is 110 which is 6 and left most 1 is for -ve sign

Calculation for Signed 4-bit Numbers

To add any two numbers using 2'sC, no matter what the sign of each is

- Do binary addition on their representations, If end carry occurs, ignore it
- Note:
 - For +ve numbers, write the binary equivalent
 - For –ve numbers, take the 2'sC of last three bits of the binary equivalent of that number and write 1 as the signed bit at the left most position

–5		1 0 1 1	–5		1 0 1 1	–5		1 0 1 1
<u>+7</u>		<u>0 1 1 1</u>	<u>+5</u>		<u>0 1 0 1</u>	<u>+3</u>		<u>0 0 1 1</u>
+2	(1)	0 0 1 0	0	(1)	0 0 0 0	–2	(0)	1 1 1 0

End Carry, ignore it

Overflow

- Occurs when the sum is out of range
- For 4-bit numbers, that range is $-8 \leq \text{sum} \leq +7$

+5	0 1 0 1	
<u>+4</u>	<u>0 1 0 0</u>	
	(0) 1 0 0 1	(looks like -7)

As 2'sC of 001 is 111 i.e. 7

Answer is clearly wrong because the correct answer (9) is out of range

Overflow

Overflow occurs whenever

- Add two positive numbers (each beginning with a 0) and get a result that looks negative (begins with a 1)
- Add two negative numbers and obtaining a sum more negative than -8

$$\begin{array}{r} -5 \qquad 1\ 0\ 1\ 1 \\ -4 \qquad 1\ 1\ 0\ 0 \\ \hline \end{array}$$

(1) 0 1 1 1 (looks like +7)

This time, two negative numbers produced a sum that looks positive.

Overflow

Addition of two numbers of the opposite sign never produces overflow, since the magnitude of the sum is somewhere between the magnitudes of the two operands

Overflow

For **signed** numbers

- End carry of high-order bit is ignored, and overflow occurs if addition process
- operates on two numbers of same sign and produces a result of the opposite sign
- Addition of a positive number to a negative one never produces overflow
- Overflow may occur if we subtract a negative number from a positive one or a positive number from a negative one

For **unsigned** numbers

- End carry of high-order bit is indicator of overflow, as in addition
- In subtraction, 0 indicates overflow and end carry of 1 indicates no overflow
- Overflow is indicated by the carry of 0

Binary Coded Decimal

- Internally, most computers operate on binary numbers. However, when they interface with humans, mode of communication is decimal
- Necessary to convert from decimal to binary on input and from binary to decimal on output
- If we use the first 10 binary numbers to represent the 10 decimal digits, then the number 739, for example, would be stored as 0111 0011 1001
- Here each decimal digit is represented by 4 bits, and thus a 3-digit decimal number requires 12 bits

Binary Coded Deciaml

Following tables summarizes various codes based on BCD

Table 1.7 Binary-coded decimal codes.

Decimal digit	8421 code	5421 code	2421 code	Excess 3 code	2 of 5 code
0	0000	0000	0000	0011	11000
1	0001	0001	0001	0100	10100
2	0010	0010	0010	0101	10010
3	0011	0011	0011	0110	10001
4	0100	0100	0100	0111	01100
5	0101	1000	1011	1000	01010
6	0110	1001	1100	1001	01001
7	0111	1010	1101	1010	00110
8	1000	1011	1110	1011	00101
9	1001	1100	1111	1100	00011
unused	1010	0101	0101	0000	any of the 22 patterns with 0, 1, 3, 4, or 5 1's
	1011	0110	0110	0001	
	1100	0111	0111	0010	
	1101	1101	1000	1101	
	1110	1110	1001	1110	
	1111	1111	1010	1111	

Binary Coded Deciaml

- In the table, the codes 8421, 5421 and 2421 are referred to as *weighted codes* because the value represented is computed by taking the sum of each digit times its weight.
- This first code is referred to as the 8421 code, since those are the weights of the bits
- Each decimal digit is represented by $8 \times a_3 + 4 \times a_2 + 2 \times a_1 + 1 \times a_0$
- It is also referred to as straight binary.
- Two other weighted codes (5421 and 2421) that are occasionally used

ASCII Codes

- Alphanumeric information is transmitted using the American Standard Code for Information Interchange (ASCII)
- Seven digits are used to represent the various characters on the standard keyboard as well as a number of control signals (such as carriage return)
- For example, the word *Logic would be coded as*

1001100 1101111 1100111 1101001 1100011

L o g i c

ASCII Code

Table 1.8 ASCII code.

$a_3a_2a_1a_0$	$a_6a_5a_4$					
	010	011	100	101	110	111
0000	space	0	@	P	`	p
0001	!	1	A	Q	a	q
0010	"	2	B	R	b	r
0011	#	3	C	S	c	s
0100	\$	4	D	T	d	t
0101	%	5	E	U	e	u
0110	&	6	F	V	f	v
0111	·	7	G	W	g	w
1000	(	8	H	X	h	x
1001	)	9	I	Y	i	y
1010	*	:	J	Z	j	z
1011	+	;	K	[	k	{
1100	,	<	L	\	l	
1101		=	M	]	m	}
1110	.	>	N	^	n	~
1111	/	?	O	_	o	delete

Gray Code

- In a *Gray code*, consecutive numbers differ in only one bit
- Table shows a 4-bit Gray code sequence

Table 1.9 Gray code.

Number	Gray code	Number	Gray code
0	0000	8	1100
1	0001	9	1101
2	0011	10	1111
3	0010	11	1110
4	0110	12	1010
5	0111	13	1011
6	0101	14	1001
7	0100	15	1000

Gray Code

- Gray code is particularly useful in coding position of continuous device
- As the device moves from one section to the next, only 1 bit of the code changes
- If there is some uncertainty as to the exact position, only 1 bit is in doubt
- If a normal binary code were used, all 4 bits would change as it moved from 7 to 8



The End